

Module: Problem 2 - Dealing with Hands

This module provides utilities for managing a “hand” of letters in word-based games (e.g., Scrabble). A hand is represented as a dictionary mapping each letter (a one-character string) to the number of times it appears. The primary operation is to consume letters from the hand when a word is played, producing an updated hand without mutating the original.

Functions

`updateHand(hand, word)`

Description:

Creates a new hand dictionary that reflects the consumption of the letters required to spell `word`. The original `hand` is left unchanged. The function assumes that the hand contains **at least** as many copies of each letter as appear in `word`; if this pre-condition is violated the behavior is undefined.

Parameters: - `hand` (*dict[str, int]*): Mapping of letters to their current counts.

- `word` (*str*): The word whose letters should be removed from the hand.

Returns:

A new dictionary (*dict[str, int]*) representing the hand after the letters of `word` have been removed. Letters whose count drops to zero are omitted from the returned dictionary.

Examples:

```
# Normal usage - all letters are present in the hand
hand = {'h': 1, 'e': 1, 'l': 2, 'o': 1}
result = updateHand(hand, 'hello')
# → {}
# `hand` is still {'h': 1, 'e': 1, 'l': 2, 'o': 1}

# Based on observed runtime behavior - passing a non-dict triggers an error
result = updateHand(0, 'hello')
# → AttributeError: 'int' object has no attribute 'copy'

result = updateHand(1, 'test')
# → AttributeError: 'int' object has no attribute 'copy'

result = updateHand(-1, '')
# → AttributeError: 'int' object has no attribute 'copy'
```

Edge Cases / Notes: - **Invalid `hand` type:** If `hand` is not a dictionary (e.g., an `int`), the function raises `AttributeError` because it attempts to call `.copy()` on the object. - **Empty `word`:** The function returns a shallow copy of `hand` (since no letters are removed). If `hand` is invalid, the same `AttributeError` occurs. - **Insufficient letters:** The implementation does **not** check that the hand contains enough of each letter; attempting to decrement a missing key will raise a `KeyError`. - **Zero-count removal:** When a letter's count reaches zero after

decrementing, the key is deleted from the result, ensuring the returned hand never contains entries with a count of zero.